SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA04
COURSE NAME: Operating Systems

COURSE OUTCOME COVERED

CO1: Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments.

QUESTIONS: 5 **Total Marks:100**

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I Priyesh Rajesh Gosavi (192511088) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

90/100

Annexure A

Short Answer Test

Question 1 (5 Marks)
Suppose a modern Operating System supports multiple users and processes with shared resources like CPU, memory, and I/O devices.
(a) Classify different types of Operating Systems suitable for this scenario and justify your choice.
(b) Draw the structure of an Operating System showing how resources are managed.
(c) Write a sequence of system calls required to create and execute a new process in such an environment. **(BL2)**

Question 2 (5 Marks)
Consider an Operating System managing multiple files and devices for concurrent users.
(a) Categorize system calls used for file and device management.
(b) Illustrate how the OS handles file operations using a layered structure diagram.
(c) Write a pseudo-code or C-based system call sequence to open, read, and close a file. **(BL3)**

Question 3 (5 Marks)
Assume a real-time Operating System used in an embedded system controlling industrial machinery.
(a) Identify the type of OS and explain why it is suitable for real-time applications.
(b) Draw a diagram representing the interaction between hardware, kernel, and user processes.
(c) List and explain system calls required for process synchronization in this system. **(BL2)**

Question 4 (5 Marks)
A distributed computing system consists of multiple interconnected computers sharing resources over a network.
(a) Classify the type of Operating System used and compare it with centralized OS.
(b) Draw the architecture of a distributed OS showing communication between nodes.
(c) Write the sequence of system calls involved in inter-process communication (IPC). **(BL3)**

Question 5 (5 Marks)
Consider a scenario where an Operating System must manage process scheduling, memory allocation, and device communication efficiently.
(a) Explain the major functions of an OS in handling these tasks.
(b) Draw a block diagram representing OS components and their interactions.
(c) Write system call examples for memory allocation and device management. **(BL3)**

RUBRICS FOR CALCULATION

Criteria	Weightage (Marks)	Level 4 – Exemplary (Full Marks)	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	35 Marks	Demonstrates clear, complete, and in-depth understanding of OS concepts (types, structure, system calls)	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answers	25 Marks	All answers are correct, precise, and relevant	Mostly correct with minor errors	Partially correct with noticeable errors	Mostly incorrect or irrelevant
Application of Knowledge	15 Marks	Effectively applies concepts to scenarios/questions	Applies with minor mistakes	Limited or partial application	No meaningful application
Completeness of Response	15 Marks	All parts of each question are fully answered	Most parts covered with small omissions	Some parts missing	Incomplete answers
Clarity & Presentation	10 Marks	Clear, well-structured, and neatly presented answers	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

(a) Classify different types of Operating Systems and justify your choice.

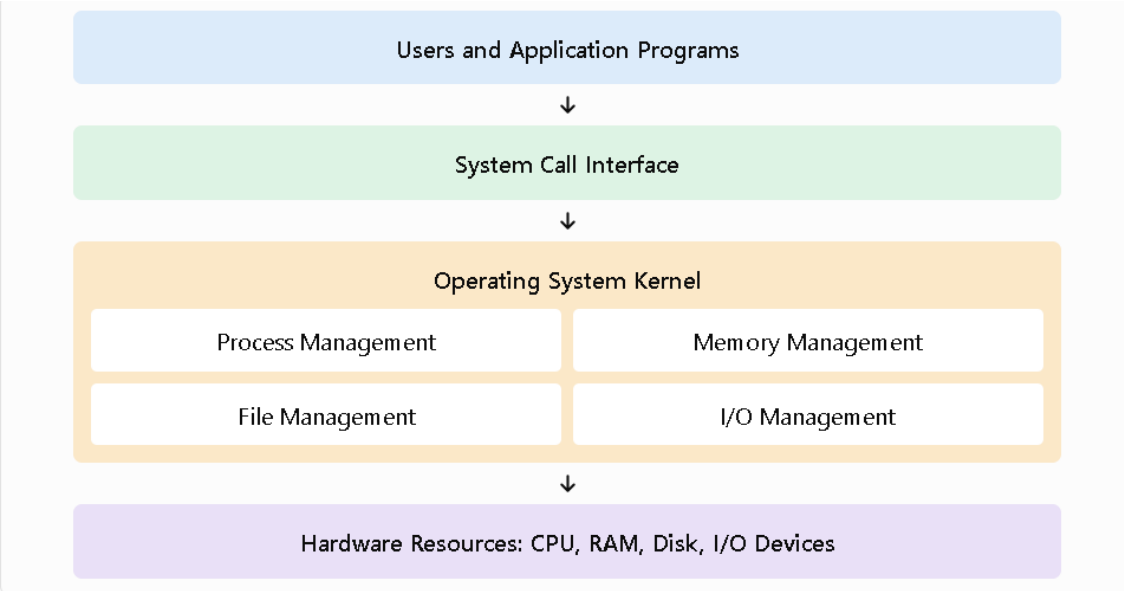
An Operating System (OS) manages hardware resources and provides services to users and applications.

The suitable types of OS for a multi-user, multiprocess environment are:

1. **Multi-user OS:** Allows multiple users to access system resources simultaneously.
2. **Multiprogramming OS**: Keeps multiple programs in memory to improve CPU utilization.
3. **Multitasking OS**: Allows multiple processes to execute concurrently through CPU scheduling.
4. **Time-sharing OS**: Allocates CPU time slices to different users and processes for interactive operation.

   Justification: A time-sharing, multi-user OS such as Linux is suitable because it supports multiple users, process scheduling, memory management, and shared I/O devices while providing controlled access to resources.

(b) Draw the structure of an OS showing how resources are managed.



Explanation:

- Process management allocates CPU time and controls processes.
- Memory management allocates and frees RAM.
- File management organizes files and directories.
- I/O management controls communication with peripheral devices.

(c) Write the sequence of system calls required to create and execute a new process.

In a Unix/Linux operating system, the sequence is:

1. **fork() –** Creates a child process**.**
2. **exec() –** Replaces the child process with a new program**.**
3. **wait() –** Allows the parent to wait for the child to finish.
4. **exit() –** Terminates the process.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec failed");
        exit(1);
    }
    else if (pid > 0) {
        wait(NULL);
        printf("Child process completed\n");
    }
    else {
        perror("fork failed");
```

```
    }

    return 0;
}
```

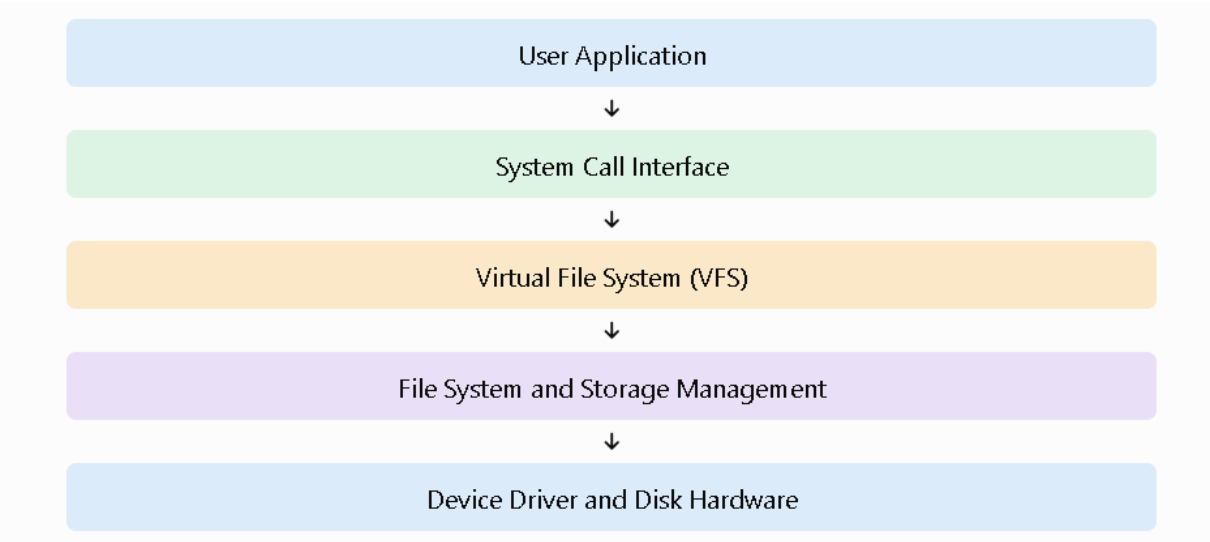
Result: The child process executes the ls -l command, and the parent waits until the child finishes.

Question 2: File and Device Management (5 Marks)

(a) Categorize system calls used for file and device management.

Category	**System calls**	**Purpose**
File management	open()	Opens a file
File management	read()	Reads file data
File management	write()	Writes data to a file
File management	close()	Closes an open file
File management	unlink()	Removes a file
Device management	ioctl()	Controls device-specific operations
Device management	open(), read(), write(), close()	Accesses device files
Device management	mmap()	Maps a file or device into memory when supported

(b) Illustrate how the OS handles file operations using a layered structure diagram.



Explanation: The application requests file operations through system calls. The OS file system manages file names, directories, permissions, and storage. Device drivers communicate with the storage hardware to complete read and write operations.

(c) Write a pseudo-code or C-based system call sequence to open, read, and close a file.

C program:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
   int fd;
   char buffer[100];
   ssize_t n;

   fd = open("input.txt", O_RDONLY);

   if (fd == -1) {
      perror("open failed");
      return 1;
   }

   n = read(fd, buffer, sizeof(buffer) - 1);
```

```
    if (n == -1) {
        perror("read failed");
        close(fd);
        return 1;
    }

    buffer[n] = '\0';
    printf("File content: %s\n", buffer);

    close(fd);
    return 0;
}
```

Result: The program opens input.txt, reads its contents, displays them, and closes the file.

Question 3: Real-Time Operating System

(a) Identify the type of OS and explain why it is suitable for real-time applications.

A Real-Time Operating System (RTOS) is designed to process data and respond to events within a specified time limit.

There are two main types:

1. Hard RTOS: Missing a deadline can cause system failure. Example: aircraft control systems.
2. Soft RTOS: Occasional deadline misses are acceptable, but performance may decrease. Example: multimedia streaming.

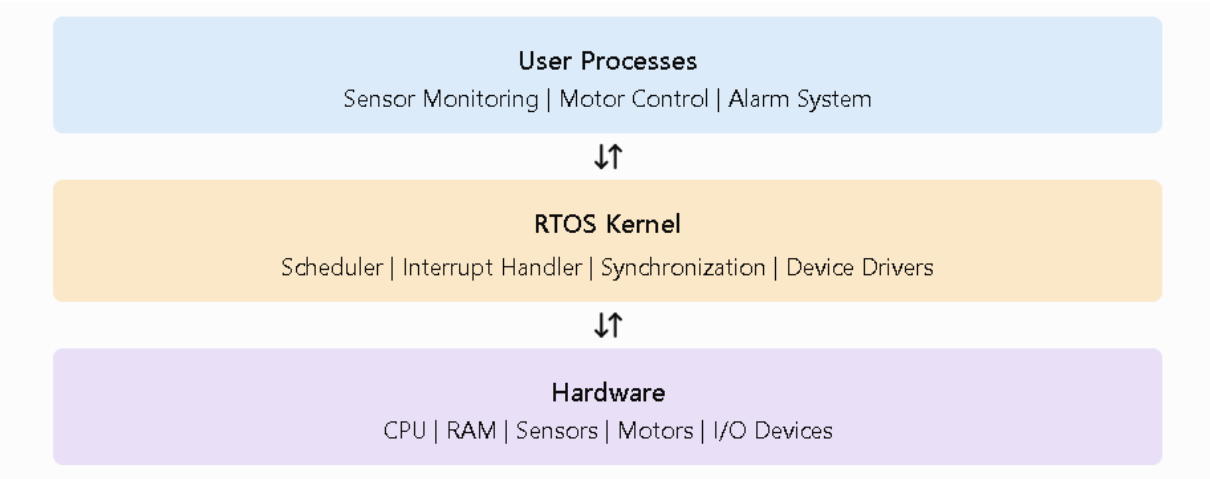
Suitable choice: A hard RTOS is appropriate for industrial machinery where delayed control signals may cause equipment damage or unsafe operation.

Examples include FreeRTOS and QNX, depending on the application requirements.

Advantages:

- Predictable response time.
- Priority-based process scheduling.
- Fast interrupt handling.
- Reliable synchronization and resource management.

(b) Draw a diagram representing the interaction between hardware, kernel, and user processes.



(c) List and explain system calls required for process synchronization.

System call / operation	Explanation
sem_wait()	Decreases a semaphore value or blocks until the resource becomes available.
sem_post()	Increases a semaphore value and may wake a waiting process.
pthread_mutex_lock()	Locks a mutex to provide exclusive access to shared data.
pthread_mutex_unlock()	Releases the mutex.
pthread_cond_wait()	Waits for a specified condition while releasing the associated mutex.
pthread_cond_signal()	Wakes one thread waiting on a condition variable.

Conclusion: Synchronization prevents race conditions and coordinates concurrent tasks.

These are POSIX synchronization functions rather than all being direct kernel system calls.

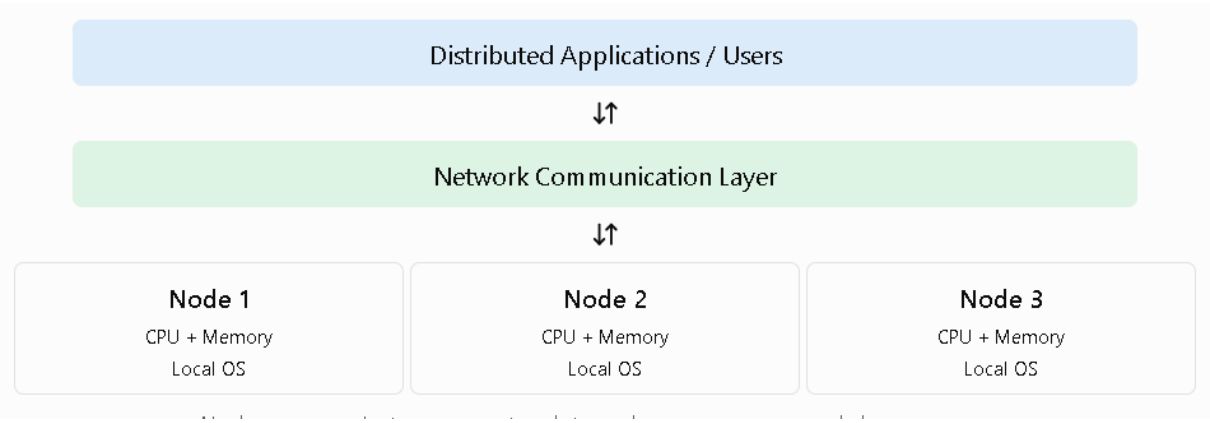
Question 4: Distributed Operating System (5 Marks)

(a) Classify the type of OS used and compare it with a centralized OS.
A Distributed Operating System manages multiple interconnected computers and coordinates their resources so that users can access services across different nodes.

Feature	Distributed OS	Centralized OS
Processing	Distributed across multiple nodes	Mainly managed by one central computer
Resource management	Coordinates resources across nodes	Manages resources at the central system
Communication	Uses network communication	Primarily uses local system communication
Scalability	Can expand by adding nodes	Expansion depends largely on central hardware
Failure handling	Some nodes may continue if another fails	Central failure may affect the entire service
Examples	Amoeba, Plan 9	Traditional single-host operating systems

Justification: A distributed OS is suitable when several computers must share processing power, files, and other resources over a network.

(b) Draw the architecture of a distributed OS showing communication between nodes.



(c) Write the sequence of system calls involved in inter-process communication (IPC).
A common IPC method is a pipe in Unix/Linux.

Sequence: pipe() → fork() → write() / read() → close() → wait().

#include <stdio.h>

```
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main() {
   int fd[2];
   char buffer[100];

   if (pipe(fd) == -1)
      return 1;

   if (fork() == 0) {
      close(fd[1]);

      int n = read(fd[0], buffer, sizeof(buffer) - 1);

      if (n > 0) {
         buffer[n] = '\0';
         printf("Child received: %s\n", buffer);
      }

      close(fd[0]);
   } else {
      close(fd[0]);

      char msg[] = "Hello from parent";
      write(fd[1], msg, strlen(msg));

      close(fd[1]);
      wait(NULL);
   }

   return 0;
}
```

Question 5: OS Functions, Components, and Resource Management

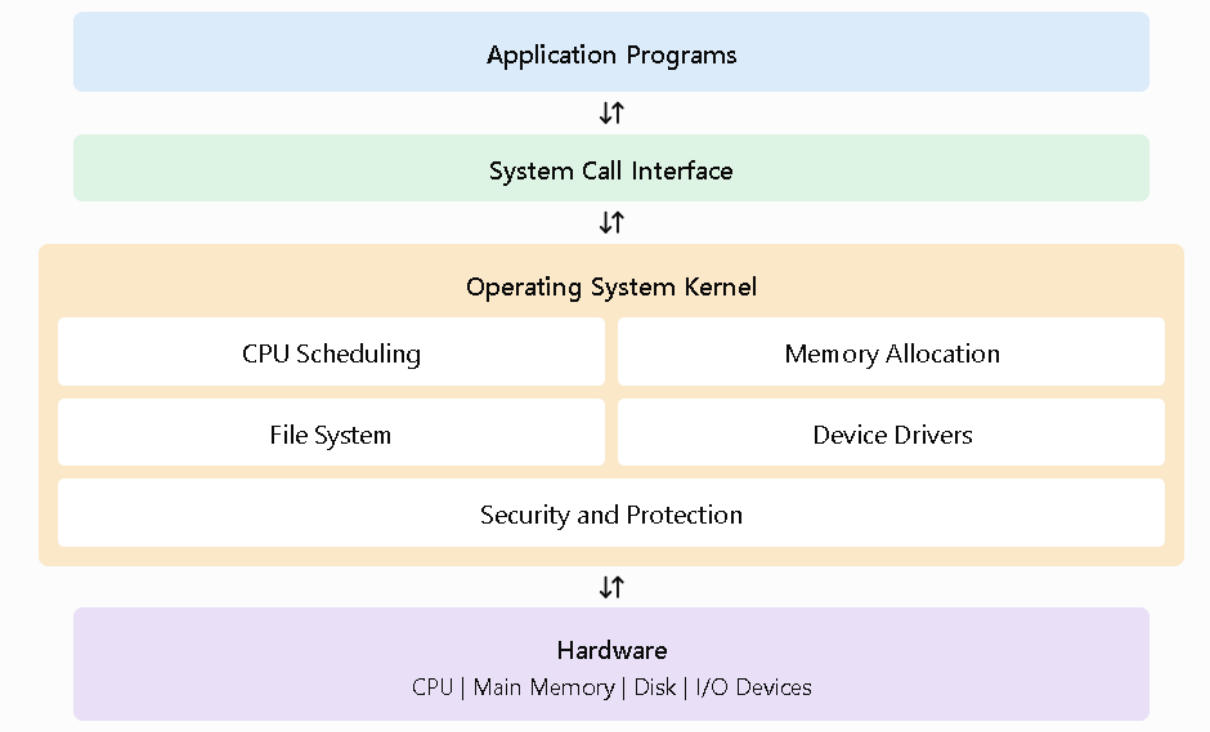
Explain the major functions of an OS in handling process scheduling, memory allocation, and device communication.

An Operating System performs the following major functions:
1. Process scheduling
   - Selects the next process to execute.
   - Allocates CPU time to processes.
   - Supports multitasking and process synchronization.
   - Uses scheduling algorithms such as FCFS, SJF, and Round Robin.
2. Memory management
   - Allocates memory to processes.
   - Tracks used and available memory.

- Frees memory when processes finish.
- Provides virtual memory and protection between processes.

3. Device management
   - Controls hardware devices through device drivers.
   - Handles input and output requests.
   - Coordinates device access between processes.
   - Uses buffering, caching, and interrupt handling where appropriate.
4. File management
   - Creates, reads, writes, and deletes files.
   - Organizes directories and controls file permissions.
5. Security and protection
   - Authenticates users.
   - Controls access to system resources.
   - Protects processes and data from unauthorized access.

(b) Draw a block diagram representing OS components and their interactions.



(c) Write system call examples for memory allocation and device management.

1. Memory allocation using mmap()

```
#include <stdio.h>
#include <sys/mman.h>

int main() {
   int *p = mmap(NULL, 4096,
              PROT_READ | PROT_WRITE,
              MAP_PRIVATE | MAP_ANONYMOUS,
              -1, 0);

   if (p == MAP_FAILED) {
      perror("mmap failed");
      return 1;
   }

   p[0] = 100;
   printf("Value = %d\n", p[0]);

   munmap(p, 4096);
   return 0;
}
```

3.Device management using open() and ioctl()

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>

int main() {
   int fd = open("/dev/example", O_RDONLY);

   if (fd == -1) {
      perror("Device open failed");
      return 1;
   }

   /* Device-specific ioctl commands
     require the appropriate driver and command. */

   printf("Device opened successfully\n");
   close(fd);
   return 0;
```

}